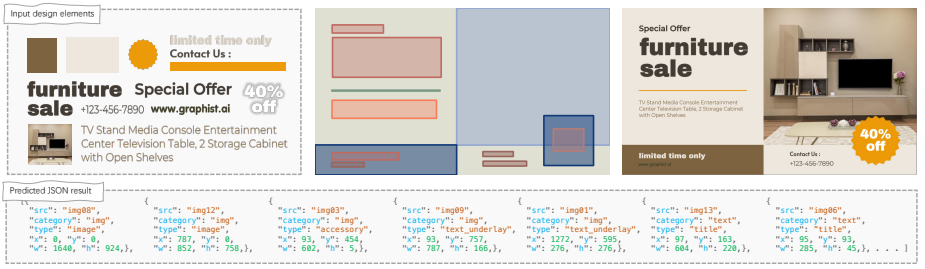

Fig. 3: A user-generated case via graphist web demo. The top-left figure represents the input design elements to Graphist. Below it, we present the corresponding output JSON code generated by Graphist. The final two images in the top row illustrate the visualized results: first is the layout visualization, and the second is the graphic composition by putting these elements according to the JSON protocol. Additional examples are available in Figure 6.

2 Related Work

2.1 Graphic Layout Generation

Graphic design is a form of visual art that combines multimodal elements (*e.g.*, images, texts, and symbols) to create aesthetically pleasing compositions which can effectively convey information to the audience. As one of the core technologies in automated graphic design, layout generation methods has been widely used in various scenarios, such as document [39, 100], UI [6, 19, 58, 82], social media snippets [90], banners [44], poster and advertisement [30, 52, 55], icon/logo [11, 24, 59, 85], CAD Sketches [27, 63], slides [26, 32, 68].

As people deepen their understanding of layout generation tasks, the modeling of layout generation problems is becoming increasingly complex. Early methods for layout generation typically relied on aesthetic rules [5, 87] or constraints [34, 37]. However, the diversity and aesthetics of generated compositions are primarily restricted by the formalized human prior knowledge, and are hard to scale up for production. Later on, some data-driven methods [2, 44, 47] emerged, utilizing learning-based generation models for layout generation. These approaches are dedicated to fitting the data distribution of the topological relationship between elements, and focus on the alignment of elements, but neglecting the content of the elements. Recent methods [12, 29, 46, 55, 92, 101] have recognized the importance of element content, and have explored ways to layout text or decorations on a given base graphic without obstructing important content. Drawing from this observation, attempts [36, 99] have also been made to incorporate multimodal inputs in order to better understand the content of each layer. These methods often assume that the layering order of the elements has been predefined [36, 86], which is difficult to achieve in the practical application. Therefore, we argue that the layer ordering needs to be fully considered in the layout generation problem.

The advancement of layout generation techniques is closely intertwined with the progress of fundamental algorithms. During the early exploration [5, 34, 37, 87], researchers attempted to define beauty using formulas and constraints. Subsequently, generative methods based on GANs [31, 47, 48], VAEs [2, 44], and diffusions [46, 82, 97] have been widely attempted. Besides, LayoutDETR [92] combining DETR [10] and generative models has been attempted to deal with the problem of element arrangement on a given image. In the era of Transformer [77], several methods [36, 39, 53] have adopted the powerful learning capabilities of Transformers to address layout and related control problems [39,53]. FlexDM [36] models graphic design using a BERT-like [20] approach. The rise of large language models [9, 75, 76] has also captured the attention of researchers in the field of graphic design, with LayoutPrompter [54] and LayoutNUWA [72] introducing language models into layout generation in a zero-shot manner. COLE [38] fuses large language models and diffusion models to build a hierarchical graphic design framework. Our approach is also inspired by large language models [76, 98] and multimodal models [13, 56, 57, 104]. However, COLE [38] focuses on utilizing the generating power of diffusion models to directly produce multi-layer graphic designs based on user intent, while our approach prioritizes the use of existing design elements for layout generation.

2.2 Large Multimodal Models

As a bridge between language and vision, LMM [45,89,95] has received widespread attention recently. Related topic including autonomous driving [17,21], video understanding [50, 71, 96], open-vocabulary vision understanding [51, 84, 93], multimodal agent [78, 88], image generation [7, 69, 94], embodied intelligence [8, 23, 60] and so on. To the best of our knowledge, Graphist is the first work to address the task of layout generation using LMM in an end-to-end manner. While COLE [38] likewise employs multimodal models in their pipeline, their main emphasis lies in generating text content and styles on predetermined base image.

Common architectures of LMM typically encompass a pre-trained visual encoder [22, 67] for extracting visual features, a pre-trained LLM [74–76] for interpreting user commands and generating responses, as well as a vision-language cross-modal connector [1, 28, 49, 57, 103] for aligning the visual encoder outputs to the language model. Given that the task of hierarchical layout generation involves organizing design elements on a blank canvas, the compatibility of coordinates is of paramount importance. Pix2Seq [15] was the first to explore the use of a discretization and serialization approach in object detection problems, in order to convert coordinates into token sequences that can ultimately be used in sequence generation mode. Some work, like OFA [79], VisionLLM [81], Kosmos-2 [33], build upon this form by introducing special coordinate tokens in their vocabulary. Alternatively, PerceptionGPT [65] implements an additional vision encoder and decoder specifically for processing and predicting coordinates. Most notable among these approaches is Shikra [13], which represents spatial positions using numerical values in natural language. It has been proven succinct and effective, as echoed by related research [4, 80, 91]. Inspired by Shikra, our

work adopts the numerical representation for coordinates in the natural language sequence.

3 Task Formulation

3.1 Graphic Layout Generation

For a unified representation, we assume that a text box can be represented by an RGB-A image, which can usually be achieved through rendering or similar methods. Under this assumption, GLG is defined as finding an optimal arrangement for a collection of RGBA design assets $\mathcal{M} = \{M_i \in \mathbb{R}^{h_i \times w_i \times 4}\}_{i=1}^{n}$. This endeavor seeks to spatially organize the set in a coherent graphic composition $\mathcal{S} = \{s_i\}_{i=1}^{n} = f(\mathcal{M})$, where $f(\cdot)$ embodies our Graphist layout methodology. Specifically, the output constitutes a set of transformations wherein for each asset M_i, a quadruple (x_i, y_i, w_i, h_i) expresses its upper-left corner placement along with the respective width and height adjustments within the composition.

3.2 Hierarchical Layout Generation

Given a set of RGB-A design elements $\mathcal{M} = \{M_i \in \mathbb{R}^{h_i \times w_i \times 4}\}_{i=1}^{n}$, HLG seeks to arrange them into a well-constructed graphic composition $\mathcal{S} = \{s_i\}_{i=1}^{n} = f(\mathcal{M})$. In this context, $f(\cdot)$ denotes the layout action implemented by our method, which we refer to as Graphist. Each element s_i in the layout prediction encompasses five numerical values $(x_i, y_i, w_i, h_i, l_i)$, associated with M_i top/left coordinates, width, height and hierarchy respectively. As shown in Figure 1, compare to conventional GLG task [86], HLG framework emphasizes the significance of element stratification by imposing a hierarchy l_i that dictates not only the placement but also the z-order of each element, thus ensuring a compositionally harmonious layering.

4 Proposed Method

4.1 Graphist Architecture

Graphist is constructed using a LMM, which receives the input of multimodal design elements and predicts a JSON fragment formatted like Figure 2. Graphist comprises three components: (i) *RGBA-Encoder*. We utilize a ViT-L/14 with 224×224 four-channel input as RGBA-Encoder, initialized with visual tower parameters of CLIP [67], excluding the alpha channel. (ii) *LLM*. Our LLM foundation incorporates Qwen1.5-0.5B/7B [3]. (iii) *Visual Shrinker*. To manage the extensive elements processing requirements of Graphist, the Visual Shrinker compresses ViT's $16 \times 16 + 1$ (cls-token) output grid feature tokens into only 5 tokens, thereby saving computational costs. Specifically, it compress the 2D output of ViT's 16×16 into 2×2 tokens via 2D average pooling and concatenates it with the cls-token to form $\mathbf{V} \in \mathbb{R}^{5 \times D}$. Consequently, it uses one